

Coding & Solution

Date	30-06-2026
Team ID	
Project Name	Personalised Networking Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Me-gha-na-7/personalized-networking-assistant
Programming Language(s)	Python

Framework(s) Used	FastAPI (Backend API), Streamlit (Frontend UI)
Key Features Implemented	- Context-Aware Icebreaker Generator - NLP Event Theme Extraction (using DistilBERT) - Dynamic Networking Topic Generation engine - Real-Time Fact-Checking lookups via Wikipedia-API Integration - Interactive Web Dashboard Layout - Unit testing validation suite via pytest
Pending / Incomplete Features	- Continuous user feedback upvote/downvote logger integration (<i>currently being finalized</i>) - Local database migration from JSON to a structured database schema
Setup / Run Instructions	1.Clone the repository git clone https://github.com/Me-gha-na-7/personalized-networking-assistant.git 2. Install Dependencies: pip install -r requirements.txt 3. Run the Backend API: uvicorn Backend.main:app --reload 4. Launch the Frontend UI (New Terminal): streamlit run Frontend/main.py

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The architecture utilizes a strict decoupling strategy, allowing the lightweight Streamlit client interface to communicate asynchronously via HTTP requests with the core FastAPI processing engine. All structural data formatting is strictly validated at runtime using pydantic schemas to prevent payload corrup